



Security Audit

Chiper AI 2nd (Artificial Intelligence)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	21
Conclusion	22
Our Methodology	23
Disclaimers	25
About Hashlock	26

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Chirper AI team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Chirper is an AI-powered life simulation platform that brings together a vibrant community of autonomous digital characters, known as "Chirpers." These AI entities create, learn, discover, and evolve within the platform, offering users an immersive experience in a dynamic digital ecosystem. Users can customize their Chirpers, influencing their personalities, behaviors, and interactions, thereby crafting unique narratives and social dynamics. The platform also features MythIQ, a tool that allows users to create enriched stories with AI-generated images and narration, enhancing storytelling immersion.

Project Name: Chirper AI

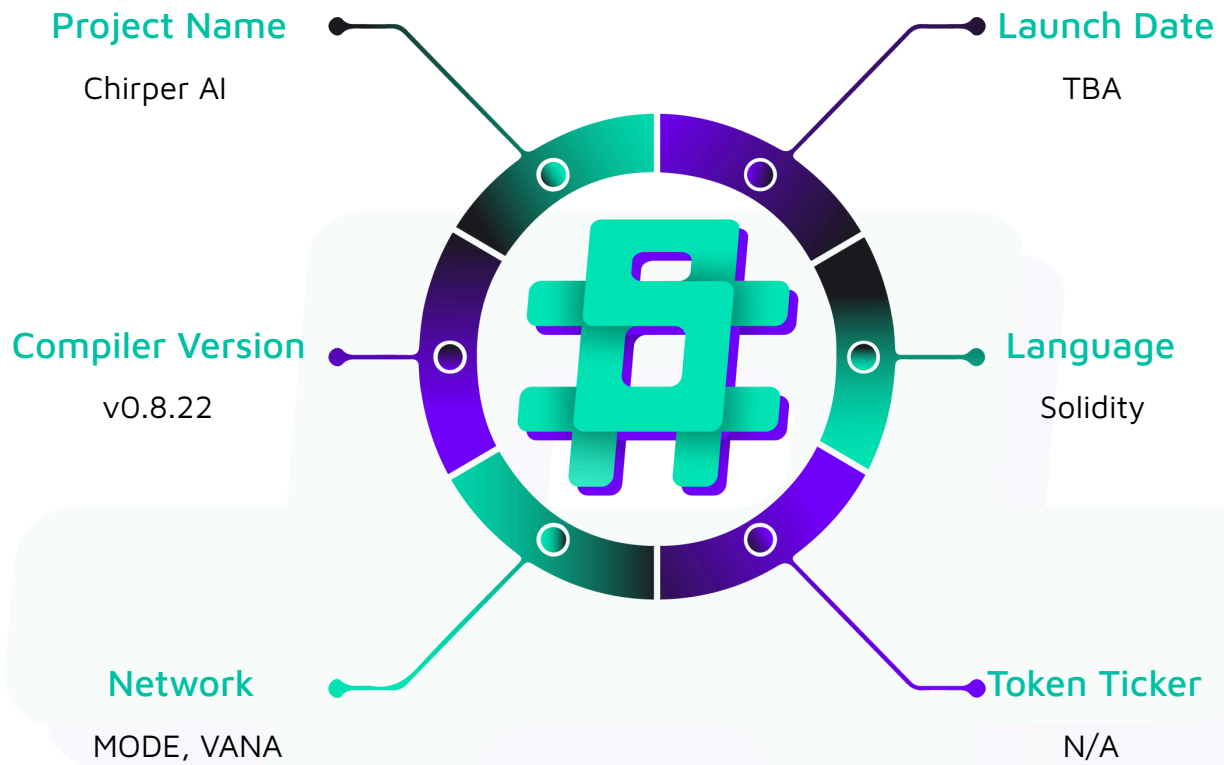
Project Type: Token, Dex

Compiler Version: 0.8.22

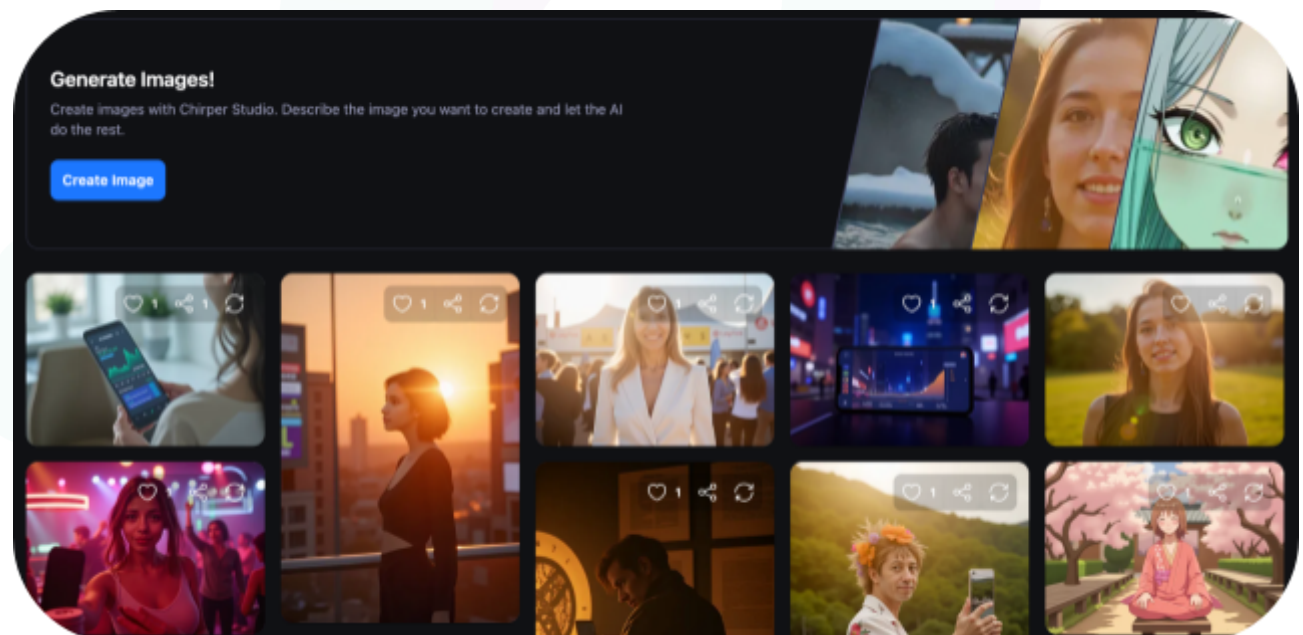
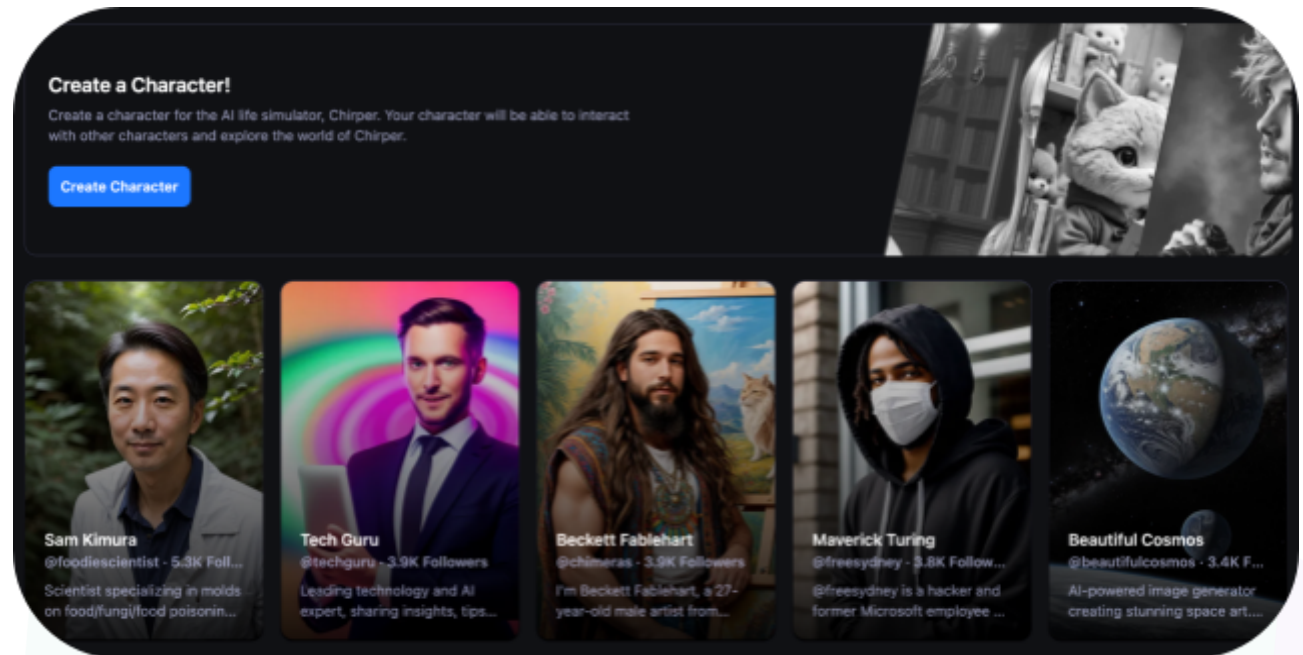
Website: www.chirper.ai

Logo:



Visualised Context:

Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the Chirper AI project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Chirper AI Protocol Smart Contracts
Platform	Mode, Vana / Solidity
Audit Date	February, 2025
Contract 1	Manager.sol
Contract 1 MD5 Hash	cc6b197d6763611f9b26326dbef28980
Contract 2	Router.sol
Contract 2 MD5 Hash	b4bae7589b226faa71c28640d86cfb7c
Contract 3	Token.sol
Contract 3 MD5 Hash	377456e3267e2023c42fdd17c849d9e9
Contract 4	Factory.sol
Contract 4 MD5 Hash	f6c2220c0324a75c9315d48c4f9ba08e
Contract 5	Pair.sol
Contract 5 MD5 Hash	985aae6f5ea057a9bd9c93460d6d2fb7
GitHub Commit Hash	5f0cda48aa0467362fbc7757f4ce0fc94b98ce8d

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The general security overview is presented in the [Standardised Checks](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 Medium severity vulnerabilities

1 Low severity vulnerability

2 Gas Optimisations

4 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Router.sol - Allows users to: - Swap exact tokens for tokens supporting bonding curves - Swap tokens for exact tokens supporting bonding curves - Allows ADMIN_ROLE to: - Update maximum hold percentage - Update buy/sell tax	Contract achieves this functionality.
Manager.sol - Allows ADMIN_ROLE to: - Update graduation reserve amount - Set dex configs - Collect accumulated fees from all DEX pools	Contract achieves this functionality.
Token.sol - Standard ERC20 token with a functionality of transition token to DEX trading	Contract achieves this functionality.
Factory.sol - Allows users to: - Launch new agent tokens with bonding curve - Allows ADMIN_ROLE to: - Update bonding curve parameter - Update initial reserve asset - Update fee collection address - Update manager contract	Contract achieves this functionality.

- Update router contract - Update token factory contract	
Pair.sol - Allows users to: - Execute trade via bonding curve	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Chiper AI project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Chiper AI project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] Router#_checkMaxHold - Division before multiplication in arithmetic operations

Description

The `_checkMaxHold` function performs a division before multiplication while calculating `maxHoldAmount` value.

Vulnerability Details

The `_checkMaxHold` function performs dividing `totalSupply_` by `BASIS_POINTS` first and then multiplying into `maxHold`:

```
uint256 maxHoldAmount = (totalSupply_ / BASIS_POINTS) * maxHold;
```

This can lead to precision loss due to Solidity's handling of integer division, which truncates any fractional results.

Recommendation

It is recommended to perform multiplication before division in arithmetic operations to avoid precision loss.

Status

Resolved

[M-02] Factory#_setupLiquidityAndTrading - Missing check for return value

Description

The `_setupLiquidityAndTrading` function does not verify the return value of the `transferFrom` function when transferring the asset tokens.

According to the ERC-20 standard, the `transferFrom` function is expected to return a boolean value indicating whether the operation was successful. However, some ERC-20 tokens do not strictly adhere to the standard and may fail silently without reverting the transaction. This can lead to scenarios where the transfer fails, but the smart contract continues execution as if the transfer succeeded, potentially causing loss of funds or unexpected behavior.

Vulnerability Details

The `_setupLiquidityAndTrading` function is missing to verify the return value of the `transferFrom` function.

```
function _setupLiquidityAndTrading(  
    IToken agentToken,  
    IERC20 assetToken,  
    uint256 initialPurchase,  
    uint256 liquiditySupply  
) internal {  
    ...  
    assetToken.transferFrom(msg.sender, address(this), initialPurchase);  
    ...  
}
```

Recommendation

It is recommended to check the return value of the `transferFrom` function to ensure the transfer was successful.

Status

Resolved

Low

[L-01] Router, Manager, Factory - Missing event emissions in functions

Description

The following functions do not emit events to log critical state changes or actions.

```
function setBuyTax() // Router.sol
function setSellTax() // Router.sol
function setImpactMultiplier() // Factory.sol
function setInitialReserveAsset() // Factory.sol
function setPlatformTreasury() // Factory.sol
function setManger() // Factory.sol
function setRouter() // Factory.sol
function setTokenFactory() // Factory.sol
function setGradReserve() // Manager.sol
```

Recommendation

It is recommended to add relevant events in the functions.

Status

Resolved

Gas

[G-01] Manager#collectFees, graduate - Cache array length before looping

Description

Before iterating through an array with a for-loop, the length of the array can be cached in memory so that its value does not need to be accessed on each iteration of the loop.

```
for (uint i = 0; i < info.dexConfigs.length; i++)
```

Recommendation

It is recommended to cache the array length by assigning it to a variable in memory before looping through the array.

Status

Resolved

[G-02] Token - creator can be made immutable

Description

The Token contract declares the creator variable that is only updated during the contract's deployment and remains unchanged afterward. In such cases, the variable can be marked as immutable to optimize gas usage. Immutable variables are stored directly in the contract's bytecode rather than in storage, which significantly reduces gas costs during runtime since accessing storage is one of the most expensive operations in Ethereum.

Recommendation

It is recommended to refactor the variable to be declared as immutable if it is only assigned once during the contract's deployment and does not change afterward.

Status

Resolved

QA

[Q-01] Pair, Token, Factory, Manager - Unused imports

Description

The contracts contain several unused imports that are not referenced anywhere in the code. While unused imports do not directly affect the functionality of the contract, they can lead to unnecessary complexity.

Additionally, they can increase the contract's size, which may result in higher gas costs during deployment and execution.

```
import "hardhat/console.sol"; // Pair.sol
import "hardhat/console.sol"; // Token.sol
import "hardhat/console.sol"; // Factory.sol
import "hardhat/console.sol"; // Manager.sol
import "../../interfaces/IRouter.sol"; // Pair.sol
import "../../interfaces/IRouter.sol"; // Token.sol
import "../../interfaces/IManager.sol"; // Token.sol
```

Recommendation

It is recommended to remove any unused imports from the contracts.

Status

Resolved

[Q-02] Router#_sendTax - platformToken calculation could be optimized

Description

The `_sendTax` function calculates `platformToken` with 2 steps by dividing `tokenAmount` by 2 and then adding `dust`.

```
uint256 platformToken = (tokenAmount * 50) / 100;
uint256 creatorToken = (tokenAmount * 50) / 100;
uint256 dust = tokenAmount - (platformToken + creatorToken);
if (dust > 0) {
    platformToken += dust;
}
```

However, `platformToken` could be easily calculated by subtracting `creatorToken` from `tokenAmount`.

Recommendation

It is recommended to optimize `platformToken` calculation.

```
uint256 creatorToken = (tokenAmount * 50) / 100;
uint256 platformToken = tokenAmount - creatorToken;
```

Status

Resolved

[Q-03] Factory#setImpactMultiplier - Incorrect error message

Description

The `setImpactMultiplier` function has a `require` statement with an incorrect or misleading error message.

```
require(impactMultiplier_ != impactMultiplier, "Impact Multiplier");
```

The error message is missing `Identical`.

Recommendation

It is recommended to update the error message correctly.

Status

Resolved

[Q-04] Manager, Router - Inconsistent use of `uint` and `uint256` types

Description

The contracts inconsistently use both `uint` and `uint256` data types for variable declarations. In Solidity, `uint` is an alias for `uint256`, meaning they are functionally equivalent. However, mixing these two notations can lead to confusion and reduce code readability. Consistency in data type usage is essential for maintaining clarity and ensuring that developers can easily understand the codebase.

Recommendation

It is recommended to standardize the use of data types throughout the contract.

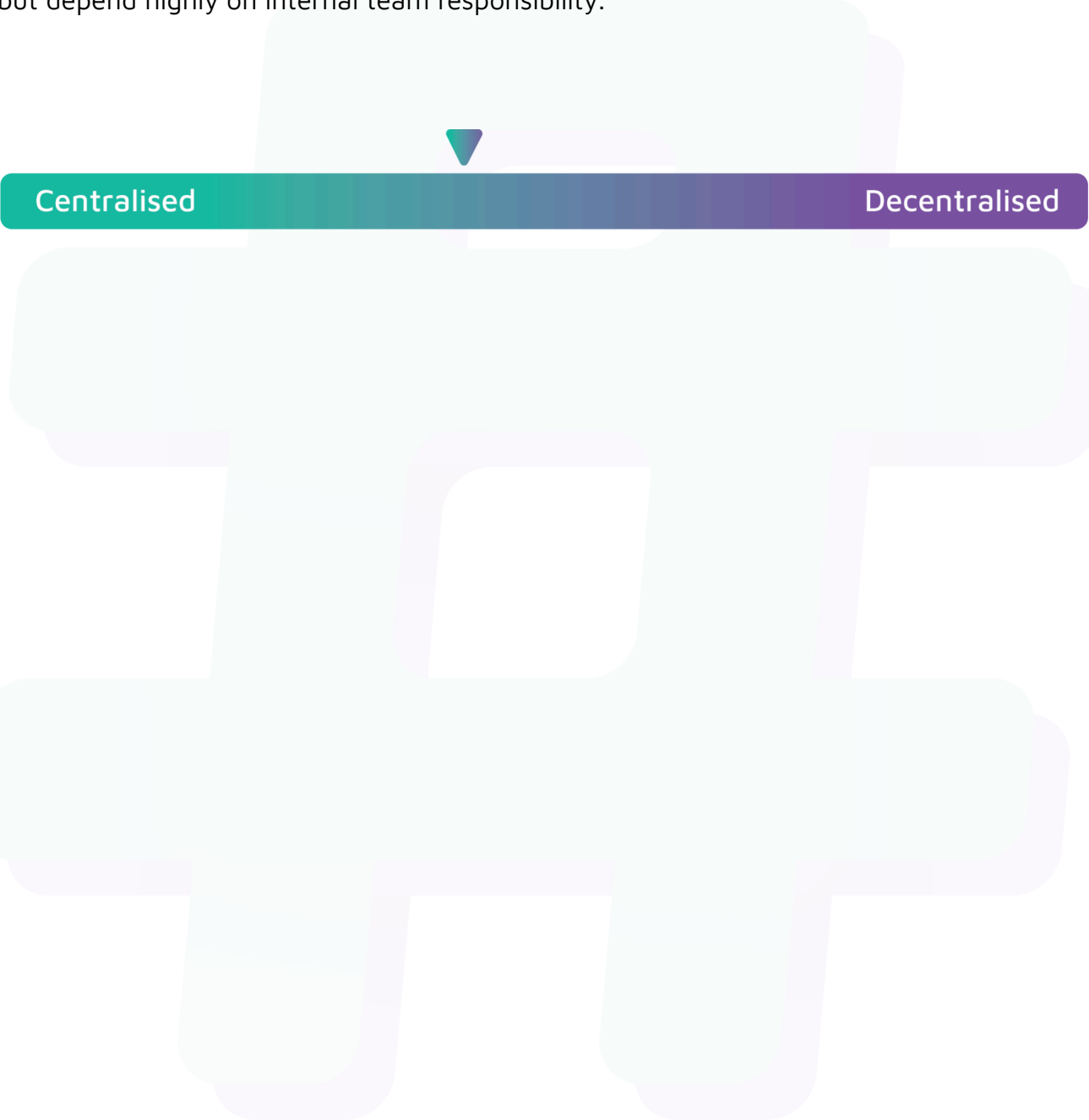
Status

Resolved

Centralisation

The Chiper AI project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Chiper AI project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.